Create two views in the database (1) CustomerView that gives Name, City and TotalAmount spent by each customer

```
MariaDB [labwork]> CREATE VIEW CustomerView AS SELECT c.customer_name AS Nam
e, c.city AS City, o.purchase_amt AS TotalAmount FROM customer c JOIN orders
 o ON c.customer_id=o.customer_id ORDER BY c.customer_name;
Query OK, 0 rows affected (0.010 sec)
```

```
MariaDB [labwork]> SELECT c.customer_name AS Name, c.city AS City, o.purchas
e_amt AS TotalAmount FROM customer c JOIN orders o ON c.customer_id=o.custom
er_id ORDER BY c.customer_name;
+--------------------+----------+-------------+
| Name               | City     | TotalAmount |
+--------------------+----------+-------------+
| Barbara Jones      | Tokyo    |     4321.11 |
| Barbara Jones      | San Jose |      670.40 |
| Barbara Lopez      | London   |     3450.60 |
| Barbara Moore      | New York |      540.25 |
| Barbara Smith      | San Jose |      535.70 |
| Charles Johnson    | Tokyo    |     2132.02 |
| Charles Jones      | Paris    |      604.53 |
| Charles Rodriguez  | Toronto  |     2962.07 |
| Charles Williams   | Toronto  |     3885.26 |
| David Brown        | Paris    |      665.31 |
| David Davis        | Tokyo    |      980.20 |
| David Jackson      | Tokyo    |     3831.23 |
```

and (2) SalesmanView that gives the Name, City and TotalAmount of sales of each sales person.

```
MariaDB [labwork]> SELECT s.name AS Name, s.city AS City, o.purchase_amt AS
TotalAmount FROM salesman s JOIN orders o ON s.salesman_id=o.salesman_id GRO
UP BY s.name ORDER BY s.name;
+-----------------+----------+-------------+
| Name            | City     | TotalAmount |
+-----------------+----------+-------------+
| Amina Abadi     | Rome     |     2953.45 |
| Arjun Mehta     | Sydney   |     3831.23 |
| Chloe Lefebvre  | Toronto  |     4435.13 |
| Diego Torres    | Tokyo    |     3885.26 |
| Elena Ricci     | San Jose |     3571.71 |
| Fatima Zahra    | Rome     |     1465.58 |
| Hans Müller     | Berlin   |     3443.09 |
| Hiroshi Sato    | Paris    |      412.04 |
| Isabella Rossi  | Paris    |     2731.00 |
| Kenji Tanaka    | Berlin   |     3782.04 |
| Lars Nielsen    | Berlin   |     3467.63 |
| Liam O'Connor   | Paris    |     3276.24 |
| Lucas Meyer     | Madrid   |      604.53 |
| Marcus Aurelius | New York |     4596.32 |
| Mateo Garcia    | Madrid   |     1812.09 |
| Oliver Twist    | Berlin   |      201.95 |
| Priya Sharma    | Rome     |     3120.99 |
| Sarah Jenkins   | London   |     3626.94 |
| Sophie Dubois   | London   |     2486.68 |
| Yuki Tanaka     | New York |     2022.77 |
+-----------------+----------+-------------+
20 rows in set (0.003 sec)

MariaDB [labwork]> CREATE VIEW SalesmanView SELECT s.name AS Name, s.city AS
 City, o.purchase_amt AS TotalAmount FROM salesman s JOIN orders o ON s.sale
sman_id=o.salesman_id GROUP BY s.name ORDER BY s.name;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MariaDB server version for the right syntax to use n
ear 'SELECT s.name AS Name, s.city AS City, o.purchase_amt AS TotalAmount FR
OM sal...' at line 1
MariaDB [labwork]> CREATE VIEW SalesmanView AS SELECT s.name AS Name, s.city
 AS City, o.purchase_amt AS TotalAmount FROM salesman s JOIN orders o ON s.s
```

```
MariaDB [labwork]> CREATE VIEW SalesmanView SELECT s.name AS Name, s.city AS
 City, o.purchase_amt AS TotalAmount FROM salesman s JOIN orders o ON s.sale
sman_id=o.salesman_id GROUP BY s.name ORDER BY s.name;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MariaDB server version for the right syntax to use n
ear 'SELECT s.name AS Name, s.city AS City, o.purchase_amt AS TotalAmount FR
OM sal...' at line 1
MariaDB [labwork]> CREATE VIEW SalesmanView AS SELECT s.name AS Name, s.city
 AS City, o.purchase_amt AS TotalAmount FROM salesman s JOIN orders o ON s.s
alesman_id=o.salesman_id GROUP BY s.name ORDER BY s.name;
Query OK, 0 rows affected (0.008 sec)

MariaDB [labwork]>
```

Apply this Function to each result row, which will create a Customer object. Add the Customer object created to an ArrayList<Customer> called customerList

```
Statement stmt = conn.createStatement();
String query = "SELECT * FROM customerview";
ResultSet rs = stmt.executeQuery(query);
while (rs.next()) {
    Function<ResultSet, Customer> rsSet = resultSet -> {
        try {
            System.out.println("in constructor");
            return new Customer(resultSet.getString( columnLabel: "Name"),
                    resultSet.getString( columnLabel: "City"),
                    Double.parseDouble(resultSet.getString( columnLabel: "TotalAmount")))
        } catch (SQLException e2){
            throw new RuntimeException("Error mapping to customer", e2);
        }
    };
    customerlist.add(rsSet.apply(rs));
    Function<ResultSet, SalesPerson> rsSet2 = resultSet ->{
        try {
            System.out.println("in constructor 2");
            return new SalesPerson(resultSet.getString( columnLabel: "Name"),
                    resultSet.getString( columnLabel: "City"),
                    Double.parseDouble(resultSet.getString( columnLabel: "TotalAmount")))
        } catch (SQLException e3){
            throw new RuntimeException("Error mapping to customer", e3);
        }
    };
    salespersonlist.add(rsSet2.apply(rs));
```

```java
public class Main {
    public static void main(String[] args) {
            }



            conn.close();
        } catch (SQLException e) {
            System.err.println("Database connection failed:");
            e.printStackTrace();
        }
        System.out.println("Question 1: print 10 customers");
        System.out.println("customers: ");
        customerlist.stream().limit( maxSize: 10).forEach(System.out::println);
        System.out.println("salespersons: ");
        salespersonlist.stream().limit( maxSize: 10).forEach(System.out::println);


    }
```

Use Collectors.averagingDouble() to get the average a Customer spent.

Print 10 Customers and 10 Salespersons.

```
C:\Users\PHOENIX\.jdks\openjdk-22\bin\java.exe "-javaagent:C:\Users\PHOENIX\OneDrive\Documents\Intell
Customer list:
Barbara Jones San Jose 670.4
Barbara Jones Tokyo 4321.11
Barbara Lopez London 3450.6
Barbara Moore New York 540.25
Barbara Smith San Jose 535.7
Charles Johnson Tokyo 2132.02
Charles Jones Paris 604.53
Charles Rodriguez Toronto 2962.07
Charles Williams Toronto 3885.26
David Brown Paris 665.31
David Davis Tokyo 980.2
David Jackson Tokyo 3831.23
David Jackson Tokyo 3420.65
David Jackson Tokyo 3759.76
```

```
C:\Users\PHOENIX\.jdks\openjdk-22\bin\java.exe "-javaagent:C:\Users\PHOENIX\OneDrive\Documents\Intell
Salesperson list:
Barbara Jones San Jose 670.4
Barbara Jones Tokyo 4321.11
Barbara Lopez London 3450.6
Barbara Moore New York 540.25
Barbara Smith San Jose 535.7
Charles Johnson Tokyo 2132.02
Charles Jones Paris 604.53
Charles Rodriguez Toronto 2962.07
Charles Williams Toronto 3885.26
David Brown Paris 665.31
David Davis Tokyo 980.2
David Jackson Tokyo 3831.23
David Jackson Tokyo 3420.65
David Jackson Tokyo 3759.76
```

Use Collectors.averagingDouble() to get the average a Customer spent

Create a list of high spenders, those who spent more than the average, ordered in decreasing amounts spent. Print the top 10 high spenders.

What city had the top 5 spenders?

Main.java × Customer.java SalesPerson.java

```java
public class Main {
    public static void main(String[] args) {

        System.out.println("Question 2: use averaging double to get average");
        //customerlist.stream().mapToDouble(n -> n.totalSpent ).average().ifPresent(System.out::println);
        //Doesnt work bc i need saved value
        double customerAvg = customerlist.stream().collect(Collectors.averagingDouble(n -> n.totalSpent));
        System.out.println(customerAvg);
        System.out.println();

        System.out.println("Question 3: create a list of top 10 high spenders");
        List<Double> highSpenders = customerlist.stream().map(n ->n.totalSpent)
                .filter(a ->a > customerAvg)
                .sorted(Comparator.reverseOrder())
                .limit( maxSize: 10).collect(Collectors.toList());
        System.out.println(highSpenders);
        System.out.println();
```

Run Main ×

```
Question 2: use averaging double to get average
2238.68

Question 3: create a list of top 10 high spenders
[4961.27, 4890.3, 4826.27, 4732.39, 4596.32, 4500.22, 4480.5, 4435.13, 4420.9, 4410.2]

Question 4: top 5 cities
[London, New York, Madrid, New York, Sydney]

Process finished with exit code 0
```

```java
        System.out.println("Question 4: top 5 cities ");
        List<String> topCities = customerlist.stream().filter(t -> t.totalSpent > customerAvg) Stream<Custom
                .sorted(Comparator.comparingDouble((Customer t) -> t.totalSpent).reversed())
                .limit( maxSize: 5)
                .map(t -> t.city) Stream<...>
                .collect(Collectors.toList());
        System.out.println(topCities);
```